

CSE 256 | Lecture 11: Pretraining: Encoders

Ndapa Nakashole

February 10, 2026

Administrative matters

- ▶ List of topics covered in the midterm is now on Canvas (Syllabus, March 3rd cell)

Recap: Transformer Architecture

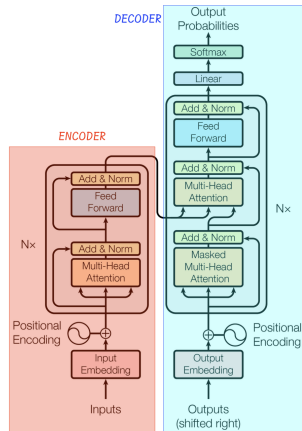
1 Attention

- Self-attention
- Multiple attention heads in parallel
- Interspersing of multi-head attention and feedforward layers
- Stacking of Transformer blocks
- Decoder self-attention (masked)
- Encoder-decoder attention (cross-attention)

2 Positional Encoding

3 Design Choices for Stable Training

- Residual connections (ResNet structure)
- Layer normalization
- Scaled dot-product attention



Today

- ① Overview of LM pretraining
- ② Representative LMs (encoder-only)
 - ELMo
 - BERT
 - DeBERTa
 - ...

Representation Learning: Text Classification Example

- **Input:** text as word embeddings for the text:

$$\{e_1, \dots, e_n\}, \quad e_i \in \mathbb{R}^d$$

- **Deep Averaging Network (DAN):** represent text by averaging word embeddings:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n e_i$$

- **Model:** feedforward neural network

$$h = f(\bar{x}) = \sigma(W \bar{x})$$

- **Output:** class label

$$\hat{y} = \text{softmax}(U h)$$

What Is Being Learned?

- ▶ Training minimizes a **supervised** loss:

$$\mathcal{L}(y, \hat{y})$$

- ▶ Parameters learned from labeled data:

$$\theta = \{\textcolor{red}{W}, \textcolor{red}{U}\}$$

- ▶ Interpretation:

- ◉ $\textcolor{red}{W}$ learns how to map documents into a **useful feature space**
- ◉ $\textcolor{red}{U}$ learns how to map those features to task-specific labels

Using Learned Representations

Given a new text sequence $\bar{\mathbf{x}}_{\text{new}}$:

$$\mathbf{h}_{\text{new}} = f(\bar{\mathbf{x}}_{\text{new}}) = \sigma(\textcolor{red}{W}\bar{\mathbf{x}}_{\text{new}})$$

- ▶ $\mathbf{h}_{\text{new}}$ is a **learned representation** for the new text
- ▶ The parameters $\textcolor{red}{W}$ define *how* representations are constructed
- ▶ The activations $\mathbf{h}$ are the representations themselves

Pretraining and Fine-tuning: Reusing Representations

🔥 **Red** = learned parameters

❄️ **Blue** = frozen parameters

- ▶ Train a model on **topic classification**, suppose we have **large labeled dataset**

Freeze and **reuse the representation**

$$h = \sigma(\text{Blue } W \bar{x})$$

- ▶ **Pretrained parameters:**

Train a new classifier on top of the frozen representation:

$$\theta_{\text{topic}} = \{\text{Blue } W, \text{Red } U_{\text{topic}}\}$$

$$\hat{y} = \text{softmax}(\text{Red } U_{\text{sentiment}} \text{Blue } h)$$

- ▶ New task: **sentiment analysis**
 - ⊙ suppose we have a **small labeled dataset**, same input structure

Two options

- ▶ **Freeze** W (feature extraction)
- ▶ **Fine-tune** W on the new task

The Key Insight

- ▶ Supervised learning in deep networks does more than just learn to map inputs to outputs
 - ◉ it learns how to represent data in a way that is useful for the task
 - ▶ These representations can be:
 - ◉ **reused** as features for new tasks
-

- ▶ **Scaling up** representation learning
 - ◉ Labeled data is **expensive**
 - ◉ Unlabeled data is **abundant**

**Can we learn powerful representations
without labeled data?**

*If so, what should the training **objective** be?*

Learning Representations: What Task Should We Train On?

What kind of training task would force a model to learn useful language representations?

- ▶ One candidate: **language modeling**
 - ⊙ predict the next word given the previous words
- ▶ To succeed at this task, a model must implicitly learn:
 - ⊙ syntax, semantics, world knowledge, ...

Learning Representations: Early OpenAI experiment

LSTM character Language Model, Trained on 82 million product reviews

Learning to Generate Reviews and Discovering Sentiment

Alec Radford¹ Rafal Jozefowicz¹ Ilya Sutskever¹

Abstract

We explore the properties of byte-level recurrent language models. When given sufficient amounts of capacity, training data, and compute time, the representations learned by these models include disentangled features corresponding to high-level concepts. Specifically, we find a single unit which performs sentiment analysis. These representations, learned in an unsupervised manner, achieve state of the art on the binary subset of

it is now commonplace to reuse these representations on a broad suite of related tasks - one of the most successful examples of transfer learning to date (Oquab et al., 2014).

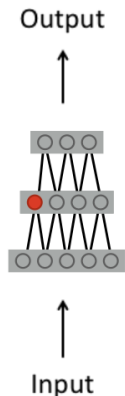
There is also a long history of unsupervised representation learning (Olshausen & Field, 1997). Much of the early research into modern deep learning was developed and validated via this approach (Hinton & Salakhutdinov, 2006) (Huang et al., 2007) (Vincent et al., 2008) (Coates et al., 2010) (Le, 2013). Unsupervised learning is promising due to its ability to scale beyond only the subsets and domains

with better coverage of target tasks. Finally, our work encourages further research into language modelling as it demonstrates that the standard language modelling objective with no modifications is sufficient to learn high-quality representations.

5 Apr 2017

What Is in a Neuron?

- ▶ A neural network contains many neurons (scalar units) a priori uninterpretable
 - ◉ Sometimes, a **single neuron can align** strongly with a **human-interpretable concept**



- ▶ Idea of **probing**: for a given neuron
 - ◉ train a simple classifier to predict a property of the input from that single neuron's activation
 - ◉ ask: does any single unit correlate with sentiment? factuality? syntax? ...

Learning Representations: the Sentiment Neuron

high $\rightarrow$ positive sentiment, low $\rightarrow$ negative sentiment

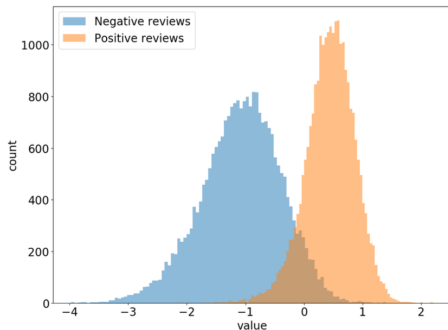


Figure 3. Histogram of cell activation values for the sentiment unit on IMDB reviews.

- **Sentiment Neuron:** a single unit which performs sentiment analysis
- **Very data efficient:** matches the performance of strong baselines trained on full datasets with only a handful of labeled examples
- 92.30% test accuracy on the binary Stanford Sentiment Treebank (PA1 dataset)

Sentiment Neuron

each review is fed into the trained language model one token at a time. After each token is processed, record the activation value of the sentiment neuron.

Once in a while you get amazed over how BAD a film can be, and how in the world anybody could raise money to make this kind of crap. There is absolutely No talent included in this film - from a crappy script, to a crappy story to crappy acting. Amazing...

Team Spirit is maybe made by the best intentions, but it misses the warmth of "All Stars" (1997) by Jean van de Velde. Most scenes are identic, just not that funny and not that well done. The actors repeat the same lines as in "All Stars" but without much feeling.

God bless Randy Quaid...his leachorous Cousin Eddie in Vacation and Christmas Vacation hilariously stole the show. He even made the awful Vegas Vacation at least worth a look. I will say that he tries hard in this made for TV sequel, but that the script is so NON funny that the movie never really gets anywhere. Quaid and the rest of the returning Vacation vets (including the original Audrey, Dana Barron) are wasted here. Even European Vacation's Eric Idle cannot save the show in a brief cameo.... Pathetic and sad...actually painful to watch....Christmas Vacation 2 is the worst of the Vacation franchise.

Figure 4. Visualizing the value of the sentiment cell as it processes six randomly selected high contrast IMDB reviews. Red indicates negative sentiment while green indicates positive sentiment. Best

Contextualized Word Representations

What does a language model learn about individual words?

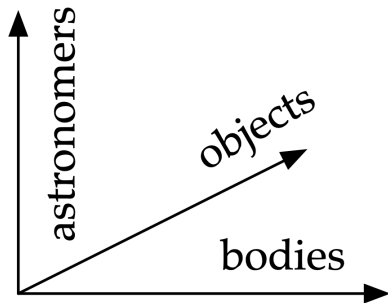
Meaning Is Really Context-Dependent

- ▶ Methods for learning word embeddings (e.g., Word2Vec)
 - ◉ Assign a **single vector per word**.
 - ◉ This leads to **conflated meanings**
- ▶ These those were inspired by distributional semantics:
"You shall know a word by the company it keeps." - (J. R. Firth, 1957)
- ▶ But the same linguistic also said:

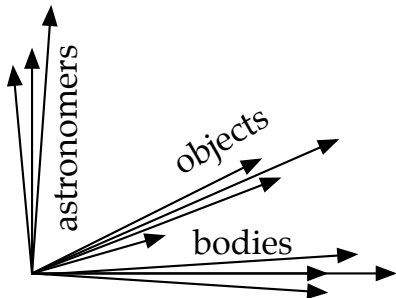
(J. R. Firth, 1935)

"The complete meaning of a word is always contextual, and no study of meaning apart from a complete context can be taken seriously."

Contextualized Word Embeddings: the dream



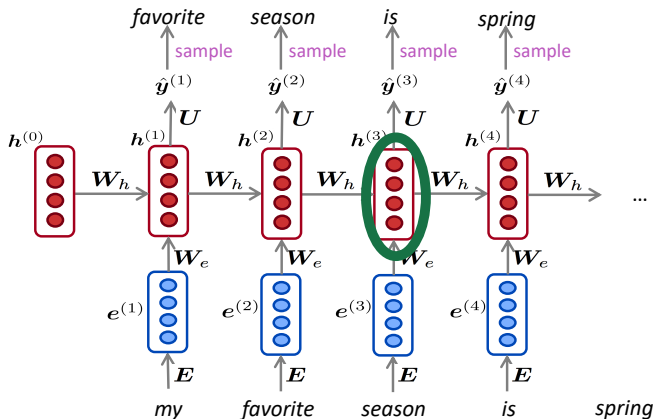
Word2Vec, Glove, etc: give a single embedding for each word regardless of context



really want: different embedding for each word in each context it appears

ELMo Insight: We have had the solution all along!

- LMs are producing context-specific word representations at each position: **hidden states** at each step (ELMo Peters et al. 2018)





- ▶ Learn word token vectors by training on language modeling objective
- ▶ Train a deep Bi-LSTM LM, freeze and use all its layers in prediction
 - ◉ Use the embeddings as a **drop-in replacement for Word2Vec**

ELMo Architecture

- ▶ **ELMo** is built using a deep **Bi-LSTM** model, where each word token's embedding is a function of:

$$\mathbf{h}_k^{LM} = [\overrightarrow{\mathbf{h}_k^{LM}}; \overleftarrow{\mathbf{h}_k^{LM}}]$$

- ▶ $\overrightarrow{\mathbf{h}_k^{LM}}$ and $\overleftarrow{\mathbf{h}_k^{LM}}$ represent the hidden states of forward and backward LSTMs for word k .

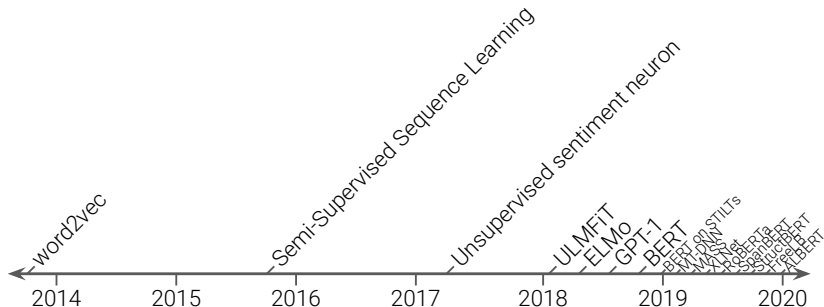
$$\mathbf{ELMo}_k = \gamma \sum_{j=0}^L s_j \mathbf{h}_{k,j}^{LM}$$

Where:

- ▶ $\mathbf{h}_{k,j}^{LM}$ is the j -th layer of the LSTM for word k .
- ▶ s_j : trainable scalar weights.
- ▶ γ : trainable scalar parameter to scale the final embedding.

AI2's ELMo came out Spring 2018: race to scale up!

- ▶ ELMo: Huge gains across many high-profile tasks: NER, question answering, etc.



BERT

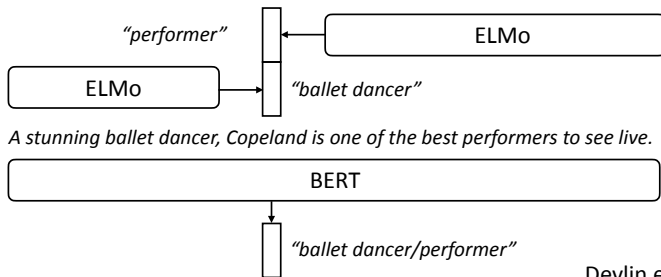
What if pretraining is **not just for the embeddings?**



- ▶ Four major changes compared to ELMo:
 - ◉ Transformers instead of LSTMs
 - ◉ Bidirectional model with "Masked Language Modeling" objective instead of standard Language Modeling
 - ◉ Fine-tune instead of freeze at test time (not just a source of word embeddings!)

BERT

- ▶ ELMo is a unidirectional model: can concatenate two unidirectional models, but is this the right thing to do?
- ▶ ELMo representations look at each direction in isolation; BERT looks at them jointly



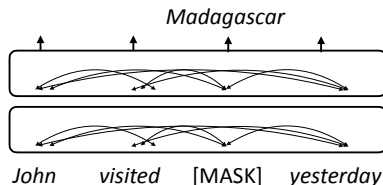
Devlin et al. (2019)

BERT: Bidirectional Context Without Cheating

- ▶ A fully bidirectional encoder can see both past and future tokens
- ▶ This makes standard language modeling impossible

Solution (Devlin et al., 2018): Masked Language Modeling (MLM)

- ▶ Randomly mask 15% of tokens
- ▶ Predict masked tokens using full context



Optimize:

$$P(\text{Madagascar} \mid \text{John visited [MASK] yesterday})$$

Pre-training BERT: Masked Language Model

Masked Language Model (MLM) is one of the two pre-training tasks in BERT.

- ▶ Given an input sentence, BERT uses the context of surrounding words to predict the masked word:

$$P(\text{word}_i \mid \text{context}) = \text{softmax}(\mathbf{W} \cdot \mathbf{h}_i)$$

- ▶ Where:
 - ◉ $\mathbf{h}_i$: The hidden state of the Transformer for word i .
 - ◉ $\mathbf{W}$: Output projection matrix.

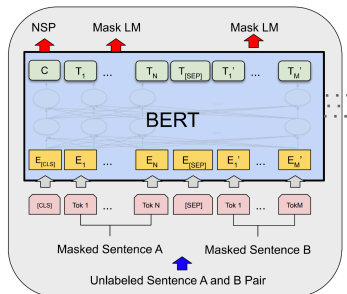
BERT Masking Strategy

store gallon
↑ ↑
the man went to the [MASK] to buy a [MASK] of milk

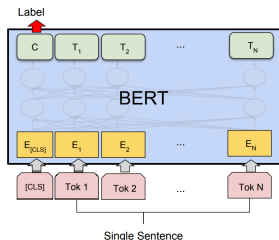
- ▶ Too little masking: training will take a long time
- ▶ Too much masking: insufficient context to infer meaning

BERT Architecture: Base and Large

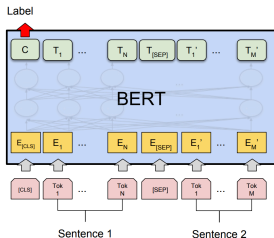
- ▶ 12/24 layers, 768/1024 hidden units, 12/16 attention heads
- ▶ 110M/340M parameters
- ▶ 512 tokens per sequence
- ▶ 16/32 sequences per batch
- ▶ Positional embeddings and segment embeddings, 30k word pieces



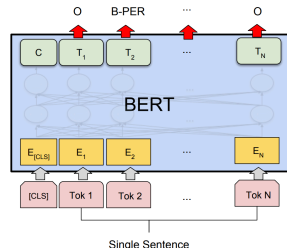
Fine-tuning BERT on Downstream Tasks



(b) Single Sentence Classification Tasks:
SST-2, CoLA



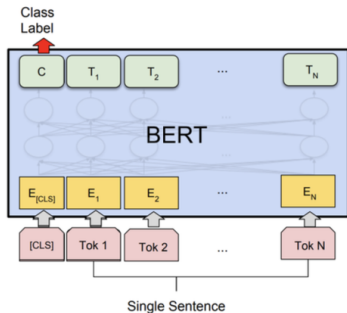
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

- ▶ Artificial [CLS] token is used as the vector to do classification
- ▶ Sentence pair tasks (entailment): feed both sentences into BERT
- ▶ BERT can also do tagging by predicting tags at each word piece

Fine-tuning BERT



(b) Single Sentence Classification Tasks:
SST-2, CoLA

- ▶ Finetune **all parameters** (both the newly added layer and the pretrained weights)
- ▶ Use a **small learning rate** (e.g., $1e-5$)
- ▶ Train for a **small number of epochs** (e.g, 3 epochs)
- ▶ Led to **SOTA results** on many tasks

Results: GLUE Benchmark

- **QQP**: Quora Question Pairs (detect paraphrase questions)
- **QNLI**: natural language inference over question answering data
- **SST-2**: sentiment analysis
- **CoLA**: corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B**: semantic textual similarity
- **MRPC**: microsoft paraphrase corpus
- **RTE**: a small natural language inference corpus

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.9	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	88.1	91.3	45.4	80.0	82.3	56.0	75.2
BERT _{BASE}	84.6/83.4	71.2	90.1	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	91.1	94.9	60.5	86.5	89.3	70.1	81.9

- ▶ Huge improvements over prior work (even compared to ELMo)
- ▶ Effective at "sentence pair" tasks

What can BERT NOT do?

- ▶ BERT **cannot generate** text (at least not in an obvious way)
- ▶ Could put [MASK] at the end repeatedly, but this is **slow** and in practice **lacks coherence**
 - ◉ **E.g.:** "The cat sat on the [MASK]" → "The cat sat on the mat"
- ▶ Masked language models are intended to be used primarily for "analysis" tasks (e.g., classification, question answering, etc.) rather than generation tasks

Encoders vs. Decoders

- ▶ BERT is a Transformer **encoder**: **bidirectional attention**, trained with masked language modeling

$$P(w_i \mid w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n)$$

- ◉ **To use in practice**: Ignore this probability distribution.

Fine-tune the model for some other task $P(\mathbf{y} \mid \mathbf{x})$

- ▶ GPT-n and other Transformer language models (e.g., PA2) are **decoders**: **unidirectional attention**, trained to predict the next word

$$P(w_i \mid w_1, \dots, w_{i-1})$$

- ◉ Can ignore this probability distribution and train a model for $P(\mathbf{y} \mid \mathbf{x})$.
- ◉ **To use in practice**: Generate text by sampling from this distribution

Why Is Decoder Generation Efficient? (and BERT is not)

- ▶ Generating with BERT (MLM) would require repeated recomputation
 - ⊙ For each new token, we would need to recompute the entire sequence representation from scratch
 - ⊙ But in autoregressive decoding, generation is faster
- ▶ In decoding, we model:

$$P(w_{1:T}) = \prod_{t=1}^T P(w_t \mid w_{<t})$$

- ⊙ At step t , the model only depends on the **prefix** $(w_1, \dots, w_{t-1})$
- ⊙ Therefore: once we have processed the prefix, it will **never change**

Decoder Hidden States?

Let the decoder produce hidden states:

$$\mathbf{h}_{t-1} = [h_1, h_2, \dots, h_{t-1}]$$

- ▶ Each h_i is a **contextual representation** of token w_i
- ▶ To compute h_t , we need access to the earlier states

Generation says: “build one more contextual vector.”

Causal Self-Attention: KV Caching

In one decoder layer, for each position i :

$$q_i = W_q h_i, \quad k_i = W_k h_i, \quad v_i = W_v h_i$$

- To compute attention for the new token t :

$$\text{Attn}(q_t, K_{1:t}, V_{1:t})$$

- **Crucial observation:** the prefix keys/values

$$K_{1:t-1}, V_{1:t-1}$$

have already been computed at the previous step

Cache K, V for past tokens (per layer). Recompute only the new token.

Why This Breaks for BERT (MLM)

- ▶ In BERT (bidirectional attention), each hidden state depends on *all* tokens
- ▶ If we replace [MASK] with a predicted token, then:
 - ⊙ the hidden state at every position changes
 - ⊙ therefore all keys/values change
- ▶ So cached (K, V) are no longer valid $\Rightarrow$ full recomputation

Caching works only when the past does not depend on the future.

How BERT Changed NLP

- ▶ BERT made **pretraining** + **fine-tuning** the *default* paradigm in NLP
- ▶ BERT showed that a single pretrained model could be:
 - ◉ dropped into many tasks
 - ◉ trained with very little labeled data, and still achieve strong performance
- ▶ As a result:
 - ◉ “design a new model per task” largely disappeared
 - ◉ Field shifted to **objectives**, **scaling**, and **efficiency**

BERT didn't just improve results, it changed how NLP was done.

BERT Variants

RoBERTa (“Robustly optimized BERT approach”)

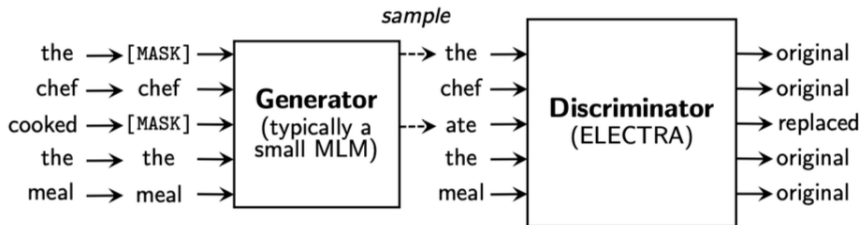
Mostly about scaling up BERT, and better training

- ▶ Train BERT with more data, longer, bigger batches, and more training steps. 160GB of data instead of 16 GB
- ▶ Dynamic masking: BERT uses the same MASK scheme for every epoch, RoBERTa recomputes them

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	94.6/89.4	90.2	96.4
BERT _{LARGE}						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7

ELECTRA

Mostly about a more data efficient pretraining objective



- ▶ BERT is inefficient, loss only computed for 15% of tokens, 85 % of tokens are wasted
- ▶ Have a prediction for every token, not just the masked ones
- ▶ Instead of predicting the masked word, predict whether a word is replaced by a different word

DeBERTa

He et al. 2021: DeBERTa: Decoding-enhanced BERT with Disentangled Attention

- Each word is **represented by two vectors**: one for **content**, one for **position**; attention weights are based on **content** and **relative positions**.

$$Q_c = HW_{q,c}, \quad K_c = HW_{k,c}, \quad V_c = HW_{v,c},$$

$$Q_r = PW_{q,r}, \quad K_r = PW_{k,r}$$

$$\tilde{A}_{i,j} = \underbrace{Q_i^c \cdot K_j^c(b)}_{\text{(a) content-to-content}}$$

$$+ \underbrace{Q_i^c \cdot K_{\delta(i,j)}^\top}_{\text{content-to-position}}$$

$$+ \underbrace{K_j^c \cdot Q_{\delta(j,i)}^\top}_{\text{(c) position-to-content}}$$

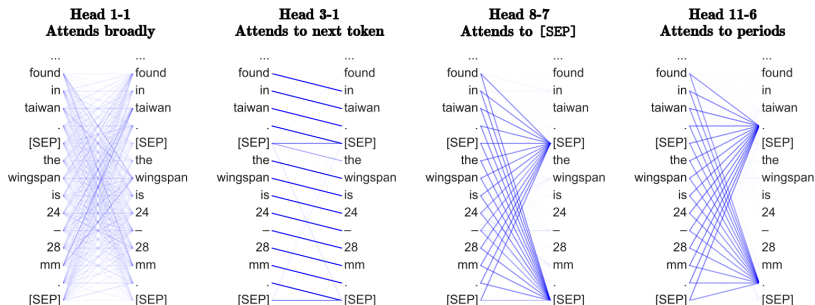
$$H_o = \text{softmax} \left(\frac{\tilde{A}}{\sqrt{3d}} \right) V_c$$

DeBERTa Results

Model	CoLA Mcc	QQP Acc	MNLI-m/mm Acc	SST-2 Acc	STS-B Corr	QNLI Acc	RTE Acc	MRPC Acc	Avg.
BERT _{large}	60.6	91.3	86.6/-	93.2	90.0	92.3	70.4	88.0	84.05
RoBERTa _{large}	68.0	92.2	90.2/90.2	96.4	92.4	93.9	86.6	90.9	88.82
XLNet _{large}	69.0	92.3	90.8/90.8	97.0	92.5	94.9	85.9	90.8	89.15
ELECTRA _{large}	69.1	92.4	90.9/-	96.9	92.6	95.0	88.0	90.8	89.46
DeBERTa _{large}	70.5	92.3	91.1/91.1	96.8	92.8	95.3	88.3	91.9	90.00

What does BERT look at?: positional and "no-op" heads

- Performance improvements suggest that pretraining teaches models about language, but **what specific linguistic features do they learn?**: Analyse the *144 attention heads* in BERT-base (12 layers x 12 heads)

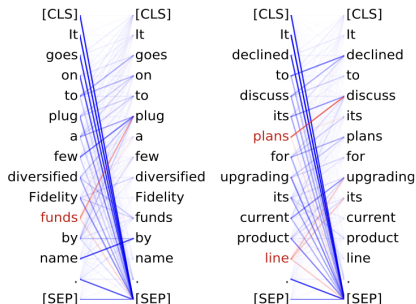


- Most heads pay **little attention on the current token**. Some heads specialize in attending to **the next or previous token**

What does BERT look at?: syntactic dependency heads

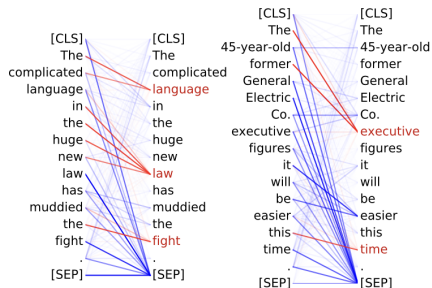
Head 8-10

- **Direct objects** attend to their verbs
- 86.8% accuracy at the dobj relation



Head 8-11

- **Noun modifiers** (e.g., determiners) attend to their noun
- 94.3% accuracy at the det relation



What does BERT look at?: syntactic dependency heads

- Different heads specialize in different aspects of syntactic relationships

Relation	Head	Accuracy	Baseline
All	7-6	34.5	26.3 (1)
prep	7-4	66.7	61.8 (-1)
pobj	9-6	76.3	34.6 (-2)
det	8-11	94.3	51.7 (1)
nn	4-10	70.4	70.2 (1)
nsubj	8-2	58.5	45.5 (1)
amod	4-10	75.6	68.3 (1)
dobj	8-10	86.8	40.0 (-2)
advmod	7-6	48.8	40.2 (1)
aux	4-10	81.1	71.5 (1)
poss	7-6	80.5	47.7 (1)
auxpass	4-10	82.5	40.5 (1)
ccomp	8-1	48.8	12.4 (-2)
mark	8-2	50.7	14.5 (2)
prt	6-7	99.1	91.4 (-1)

Pretraining Data and Models

What to Train on?

"... commercial language models rarely provide any information about their data; even open models rarely release datasets they are trained on, or an exact recipe to reproduce them. "

: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research

Luca Soldaini^{♥α} Rodney Kinney^{♥α} Akshita Bhagia^{♥α} Dustin Schwenk^{♥α}
David Atkinson^α Russell Authur^α Ben Bogin^{αω} Khyathi Chandu^α
Jennifer Dumas^α Yanai Elazar^{αω} Valentin Hofmann^α Ananya Harsh Jha^α
Sachin Kumar^α Li Lucy^β Xinxu Lyu^ω Nathan Lambert^α Ian Magnusson^α
Jacob Morrison^α Niklas Muennighoff^α Aakanksha Naik^α Crystal Nam^α
Matthew E. Peters^σ Abhilasha Ravichander^α Kyle Richardson^α Zejiang Shen^τ
Emma Strubell^{χα} Nishant Subramani^{χα} Oyvind Tafjord^α Pete Walsh^α
Luke Zettlemoyer^ω Noah A. Smith^{αω} Hannaneh Hajishirzi^{αω}
Iz Beltagy^α Dirk Groeneveld^α Jesse Dodge^α
Kyle Lo^{♥α}

^αAllen Institute for AI ^βUniversity of California, Berkeley ^χCarnegie Mellon University
^σSpiffy AI ^τMassachusetts Institute of Technology ^ωUniversity of Washington

{lucas,kylel}@allenai.org

[cs.CL] 31 Jan 2024

Dolma: 3T Tokens

Source	Doc Type	UTF-8 bytes (GB)	Documents (millions)	Unicode words (billions)	Llama tokens (billions)
Common Crawl	 web pages	9,022	3,370	1,775	2,281
The Stack	 code	1,043	210	260	411
C4	 web pages	790	364	153	198
Reddit	 social media	339	377	72	89
PeS2o	 STEM papers	268	38.8	50	70
Project Gutenberg	 books	20.4	0.056	4.0	6.0
Wikipedia, Wikibooks	 encyclopedic	16.2	6.2	3.7	4.3
Total		11,519	4,367	2,318	3,059

Using BERT, DeBERTa, ...

HuggingFace Transformers: big open-source library with most pre-trained architectures implemented, weights available

Lots of standard models...

and "community models"

Model architectures

😊 Transformers currently provides the following NLU/NLG architectures:

1. **BERT** (from Google) released with the paper [BERT: Pre-training of Deep Understanding](#) by Jacob Devlin, Ming-Wei Chang, Kenton Lee and Krist
2. **GPT** (from OpenAI) released with the paper [Improving Language Under Radford, Karthik Narasimhan, Tim Salimans and Ilya Sutskever.](#)
3. **GPT-2** (from OpenAI) released with the paper [Language Models are Un Jeffrey Wu*, Rewon Child, David Luan, Dario Amodei** and Ilya Sutskever.](#)
4. **Transformer-XL** (from Google/CMU) released with the paper [Transform Fixed-Length Context](#) by Zihang Dai*, Zhilin Yang*, Yiming Yang, Jaime
5. **XLNet** (from Google/CMU) released with the paper [XLNet: Generalized Understanding](#) by Zhilin Yang*, Zihang Dai*, Yiming Yang, Jaime Carbon
6. **GLM** (from Facebook) released together with the paper [Cross-lingual L and Alexis Conneau.](#)
7. **RoBERTa** (from Facebook), released together with the paper a [Robustly](#)

[mrm8488/spanbert-large-finetuned-tacred](#) ★

[mrm8488/xlm-multi-finetuned-xquadv1](#) ★

[nlpaueb/bert-base-greek-uncased-v1](#) ★

[nlptown/bert-base-multilingual-uncased-sentiment](#) ★

[patrickvonplaten/reformer-crime-and-punish](#) ★

[redewiedergabe/bert-base-historical-german-rw-cased](#) ★

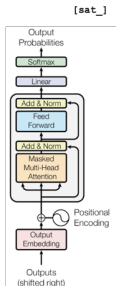
[roberta-base](#) ★

[severinsimmler/literary-german-bert](#) ★

[seyonec/ChemBERTa-zinc-base-v1](#) ★

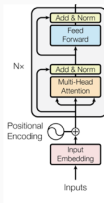
Next: decoders and encoder-decoder models

Decoder-only GPT



Encoder-only BERT

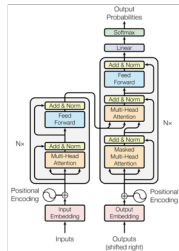
[*] [*] [sat_] [*] [the_] [*]



[The_] [cat_] [MASK] [on_] [MASK] [mat_]

Enc-Dec T5

Das ist gut.
A storm in Attala caused 6 victims.
This is not toxic.



Translate EN-DE: This is good.
Summarize: state authorities dispatched..
Is this toxic: You look beautiful today!

Takeaways

- ▶ BERT and its variants are powerful models for a range of NLP tasks
- ▶ These models have enabled big advances in NLI and QA specifically
- ▶ Next: pre-trained decoders (GPT-i) and encoder-decoder models (T5)

Getting Started in NLP Research

Old Deep Learning for NLP vs New Deep Learning for NLP

2010-2018: Early Deep Learning Revival

- ▶ **Focus:** Defining and exploring better deep learning architectures.
- ▶ E.g., Paper: A new summarization architecture that uses **copying attention**, a probabilistic gate is added to allow the model to directly copy words from the input to the output (Abigail See et al., ACL 2017).

2019-now: Modern Deep Learning in NLP

- ▶ **Focus:** Fine-tuning pre-trained models (which fixes the architecture).
- ▶ The action has shifted from architecture design to optimizing for specific tasks and domains.

current NLP ... recommended starting point

pip install transformers # By Huggingface 🤗

not quite runnable code but gives the general idea....

```
from transformers import BertForSequenceClassification, AutoTokenizer
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
model.train()
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
fine_tuner = Trainer( model=model, args=training_args, train_dataset=train_dataset,
    eval_dataset=test_dataset )
fine_tuner.train()
eval_dataset = load_and_cache_examples(args, eval_task, tokenizer, evaluate=True)
results = evaluate(model, tokenizer, eval_dataset, args)
```

Scaling up and down

Scaling models up and down is a key focus in NLP research.

► Scaling models up:

- ⊙ Building large models is impactful but requires substantial compute resources.
- ⊙ **be realistic** about the compute you can access.

► Scaling models down:

- ⊙ Building small, performant models is equally valuable. This could be a great project idea!
- ⊙ Techniques include **model pruning** and **model quantization**.
- ⊙ How well can you do QA in a **6GB** or **500MB** model? See [EfficientQA](#).

Getting Started in NLP ideas

This is not an exhaustive list, but a starting point for ideas.

1 Task/Application Exploration:

- Find a task of interest and explore how to solve it, often with an existing model.
- This could involve a real-world task, or **Kaggle** competition or **shared task** dataset.

2 Complex Neural Architecture Implementation:

- Implement a complex (perhaps novel) neural architecture and explore its performance on some dataset.

3 Prompting or In-context Learning:

- Explore the use of **prompting** of LLMs, find effective prompts, etc
- More on this in another lecture

Getting Started in NLP ideas continued

4 Model Adaptation:

- Adapt a model to a new task or domain, or explore how to make a model more robust to certain types of errors or inputs (more on this ...)

5 Data Augmentation:

- Explore how data augmentation can improve model performance, or investigate the types of augmentation that are most effective.

6 Parameter-efficient fine-tuning:

- Techniques like **low-rank adaptation** and **parameter-efficient fine-tuning**
- more to come

7 Ethical/Social Impact:

- Investigate the ethical or social implications of a model or application, or explore ways to mitigate bias or other ethical concerns.

That's it!